Distributed Systems Project: Permissioned Blockchain Inventory System

Cancian Matteo **Del Pio Luogo Marco**

Version 0.2 — August 11, 2026

Abstract

Managing inventory across multi-site enterprise environments presents critical challenges regarding data integrity, traceability, and accountability. Traditional centralized systems introduce single points of failure and rely on trusted administrators who can unilaterally modify audit logs. This project proposes a fault-tolerant, permissioned blockchain-based distributed system for enterprise asset tracking. The system guarantees immutable, append-only logging of inbound and outbound inventory transactions while supporting human error remediation through cryptographic compensating operations. Operating over a network with fair-loss links under a synchronous model, the platform leverages Proof of Authority (PoA) and Byzantine Fault Tolerance (BFT) consensus to achieve deterministic block finality, high throughput, and resilience against adversarial node behavior.

Contents

Chapter 1

Introduction

1.1 Application Overview and Problem Statement

In modern enterprise environments operating across geographically distributed sites (like national or continental logistics hubs), managing inventory with high reliability is a primary operational requirement. Organizations must track every item movement seamlessly, covering two fundamental workflows:

- **Inbound Operations:** Recording newly arrived goods or asset creations into the global inventory system.
- **Outbound Operations:** Documenting the dispatch, sale, or permanent removal of assets from the inventory.

Centralized inventory databases suffer from fundamental vulnerabilities and limitations. A compromised system administrator or external attacker can alter historical logs, modify asset quantities retroactively, or erase operational records without detection. Furthermore, centralized infrastructures represent Single Points of Failure (SPF) vulnerable to hardware faults, network partitioning, and physical disasters.

Conversely, human operators and automated IoT scanners inevitably commit operational errors (misscanning barcodes, miskeying quantities, or logging duplicate items). In an immutable ledger, simply deleting or modifying past entries is forbidden by design. So, a robust distributed inventory system must provide a mechanism for *error remediation* through explicit, auditable compensating transactions without compromising historical integrity.

1.2 Proposed Solution: Permissioned Blockchain Architecture

To address these challenges, we design a distributed inventory management platform backed by a **Permissioned (Consortium) Blockchain**. Unlike public permissionless blockchains (e.g., Bitcoin or Ethereum), where unknown participants compete via energy-intensive Proof of Work (PoW), a permissioned blockchain restricts ledger validation to pre-authorized enterprise nodes.

The system relies on two primary node types:

1. **Validator/Blockchain Nodes:** Enterprise servers deployed at key regional headquarters or trusted data centers. They maintain the full replicated ledger, run distributed consensus protocols, validate incoming transaction payloads, and assemble transactions into sequential blocks.
2. **Client/Operator Nodes:** User-facing client terminals (web frontends or handheld devices) utilized by warehouse staff. Client nodes do not participate in block generation;

instead, they sign transactions locally and submit them to validator nodes via API endpoints.

By storing item movements as cryptographically signed transaction logs linked through cryptographic hashes ($H(B_n) = \text{Hash}(B_{n-1} \parallel T_n \parallel \text{nonce})$), the system ensures strict sequential ordering, immutability, and full accountability.

In this project, we omitted the implementation of **strong authentication methods** for the 'operator'. We therefore assume that transactions are authentically signed by the operators, making it impossible for a Byzantine node to alter them.

1.3 Trade-off Analysis of Blockchain Technology

Applying blockchain technology to enterprise asset management introduces distinct advantages alongside technical trade-offs, summarized below.

Advantages

- **Immutability and Anti-Tampering:** Once a block is committed through consensus, retroactively modifying a transaction requires re-executing consensus across the majority of validator nodes, making fraud virtually impossible.

- **Complete Historical Traceability:** The chain provides an unbroken audit log for every asset from initial registration (inbound) to final disposal (outbound), eliminating operational disputes between warehouse sites.

- **High Infrastructure Resilience:** Replicating the state across independent regional validator nodes guarantees high availability; the failure or destruction of a single site does not cause data loss or system downtime.

Disadvantages and Mitigations

- **Error Remediation Complexity:** Because records cannot be erased, mistake correction requires introducing new explicit compensating transactions (e.g., explicit reversals) linked to the original erroneous transaction ID.

- **Latency vs Throughput:** Distributed consensus introduces message overhead compared to traditional relational databases. Using Proof of Authority (PoA) minimizes this performance cost.

1.4 Consensus Mechanism: Proof of Authority (PoA)

Because all validator nodes represent known corporate entities, consensus is achieved via **Proof of Authority (PoA)** and leader-based Byzantine Fault Tolerance (BFT) algorithms (e.g., PBFT).

In PoA, round-robin or stake-weighted validator rotations designate a block proposer for each time interval. The proposer aggregates pending transactions, signs the proposed block, and broadcasts it to peer validators. Peer nodes verify the proposer's digital signature and the validity of all contained transactions before appending the block to their local copy of the ledger.

1.5 Testing Plan and Project Schedule

Testing Strategy

- **Unit Testing:** Verification of modules and functions independently through specific test suites.
- **Integration Testing:** Testing client-to-validator communication, transaction propagation, and state machine transitions.
- **System & Fault Tolerance Testing:** Simulating node crashes, network packet loss, message reordering, and malicious/Byzantine transaction proposals.

Project Schedule

The project design and implementation follow a four-stage timeline:

1. **Phase 1 (Requirements & Architecture):** Defining system specifications, network models, and UML sequence diagrams.
2. **Phase 2 (Core Development):** Implementing the permissioned blockchain node, networking functionlities, and basic consensus mechanism.
3. **Phase 3 (Client Layer):** Developing client APIs, operator UI, and compensating transaction handling.
4. **Phase 4 (Validation & Evaluation):** Conducting benchmark simulations, network failure injection, and report finalization.

Chapter 2

Analysis

In this chapter, we outline the functional and non-functional requirements for the proposed blockchain inventory system.

2.1 Functional Requirements

FR-1: Inbound Inventory Registration
- **Description:** Allows an authorized operator to register new assets into the global inventory.
- **Input:** Asset details (ID, description, batch number, quantity, location), operator digital signature.
- **Output:** Transaction hash and confirmation of block inclusion.
- **Effect:** Increases global asset count; creates initial cryptographic provenance record on the ledger.

FR-2: Outbound Inventory Tracking
- **Description:** Registers the dispatch or removal of existing assets from stock.
- **Input:** Asset ID, quantity removed, destination details, operator digital signature.
- **Output:** Transaction receipt or error notification (if insufficient stock exists).
- **Effect:** Decreases available asset count; appends outbound movement record to the chain.

FR-3: Error Remediation (Compensating Transactions)
- **Description:** Corrects operational errors without overwriting historical records.
- **Input:** Target erroneous Transaction ID, correction type (e.g., reversal, adjustment), reason string, supervisor signature.
- **Output:** Remediation transaction receipt linked to the original transaction ID.
- **Effect:** Updates current inventory state while maintaining a complete audit trail of the original error and its correction.

FR-4: State Querying and Historical Audit
- **Description:** Enables operators and auditors to inspect current stock levels or trace an asset's complete lifecycle.
- **Input:** Asset ID or time interval query parameters.
- **Output:** Formatted list of chronological historical blocks and state transitions associated with the asset.
- **Effect:** Read-only execution; does not alter state.

FR-5: User Authentication and Access Control
- **Description:** Enforces Role-Based Access Control (RBAC) via public-key cryptography.
- **Input:** User private key / cryptographic certificate.
- **Output:** Authenticated session token and authorization rights.
- **Effect:** Restricts transaction creation to validated warehouse personnel and supervisors.

2.2 Non-Functional Requirements

NFR-1: Security & Cryptographic Integrity All client-to-node and node-to-node communications must be encrypted using TLS 1.3. Transactions must be signed using ECDSA (or Ed25519). Blocks are cryptographically linked using SHA-256 hash digests.

NFR-2: Availability & Fault Tolerance The consensus algorithm must tolerate up to f failing or Byzantine nodes in a system of n validator nodes, where $n \geq 3f+1$ for BFT schemes, or $n \geq 2f + 1$ for crash fault tolerant (CFT) variants under PoA.

NFR-3: Consistency & Finality The distributed ledger must deliver deterministic block finality (i.e., zero fork probability once a block is confirmed by a supermajority of validators), ensuring immediate consistency across all warehouse locations.

NFR-4: Performance & Throughput The system must process a minimum of 100 transactions per second (TPS) with block creation intervals set below 2 seconds, ensuring operational responsiveness for warehouse operators.

NFR-5: Auditability & Transparency The full history of all operations must be verifiable by any authorized validator node without relying on a centralized central authority.

Chapter 3

Project

This chapter describes the system models, logical architecture, protocols, and deployment structure.

3.1 System and Network Model

To formally reason about correctness, fault tolerance, and consensus guarantees, we define our system parameters across three dimensions:

- **Network Model (Fair-Loss Links):** Communication links between nodes are assumed to be fair-loss. Messages may be lost, duplicated, or reordered during transmission. However, if a sender retries transmitting a message infinitely often, it will eventually be delivered to the destination.
- **Node Model (Adversarial / Byzantine):** Validator nodes can exhibit arbitrary (Byzantine) behavior. A compromised or faulty node may crash, delay messages, drop valid transactions, forge signatures, or send conflicting messages to different peers.
- **Timing Model (Synchronous):** Message propagation latency across the network is bounded by a known constant Δ. Nodes operate with bounded execution speed drift, allowing the system to utilize timeout-based round rotas for consensus leader selection.

3.2 Logical Architecture

The system architecture is structured into three logical layers (Figure **??**):

1. **Presentation / Client Layer:** Consists of client software operated by warehouse workers ($\text{Operator}_1, \text{Operator}_2, \dots$). It manages local key pairs, captures user input, constructs transaction structures, signs payloads, and submits them to accessible validator nodes.
2. **Middleware / API Gateway Service:** Validates payload structure, verifies user signatures against known public key registries, checks account permissions, and forwards valid pending transactions into the memory pool (mempool).
3. **Blockchain Consensus Core:** The decentralized core comprising interconnected validator servers ($\text{Server}_X, \text{Server}_Y, \text{Server}_Z$). Each server houses:
    - *Mempool Manager:* Holds pending, unconfirmed transactions.
    - *Consensus Engine:* Executes the Proof of Authority / BFT protocol.
    - *State Machine Replication (SMR) Module:* Applies verified blocks to update local inventory state tables.

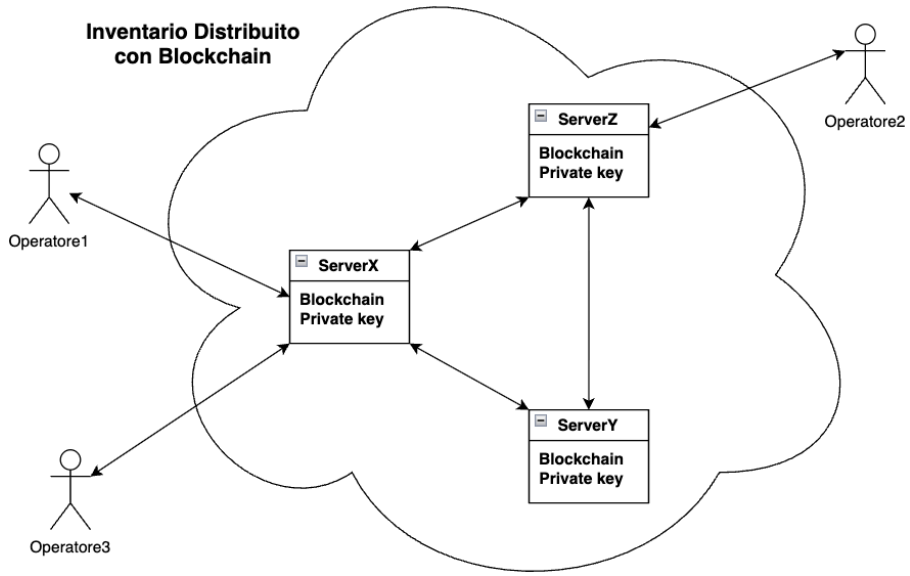


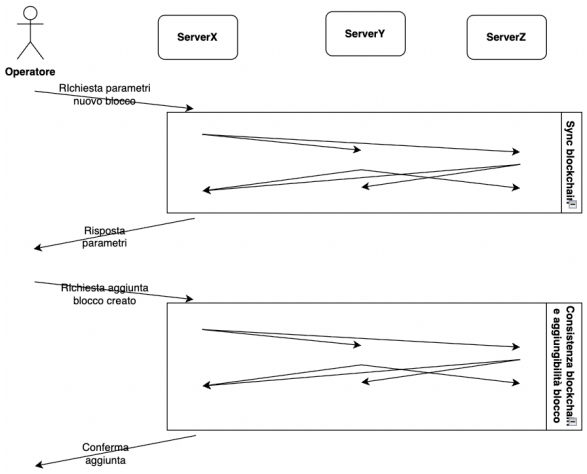
Figure 3.1: Logical Architecture and Component Interactions of the Distributed Ledger System.

- *Ledger Storage:* Append-only disk storage containing linked blocks and indexed databases (e.g., LevelDB/RocksDB) for fast state queries.

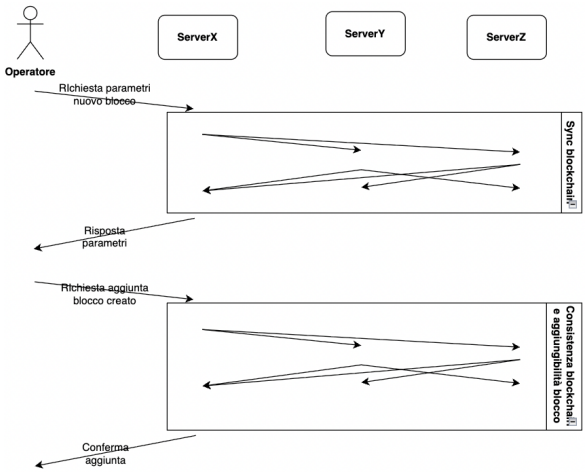
3.3 Protocols and Algorithms

Transaction Lifecycle and Block Proposal

The step-by-step operational flow for inventory updates is illustrated in Figure **??**:



(a) Sequence Diagram: Transaction Submission & Validation.



(b) Sequence Diagram: Consensus Block Propagation.

Figure 3.2: UML Sequence Diagrams outlining component interactions during operation execution.

1. **Submission:** Client node drafts transaction $T = \langle \text{Type}, \text{AssetID}, \text{Qty}, \text{Timestamp}, \text{Sig}_{\text{Op}} \rangle$ and sends it to Server_X.

2. **Broadcast & Pooling:** Server_X validates Sig_{Op}, checks local inventory constraints, and broadcasts T to peer nodes ($\text{Server}_Y, \text{Server}_Z$).

3. **Block Assembly:** The round leader collects pending transactions from its mempool into block candidate $B_k = \langle k, \text{Hash}(B_{k-1}), \{T_1, \ldots, T_m\}, \text{Sig}_{\text{Leader}} \rangle$.

4. **Consensus Vote:** The leader broadcasts B_k. Peers verify signatures, hash chain continuity, and contained transactions. If valid, peers return a pre-commit vote.

5. **Commit & State Update:** Upon gathering a supermajority of valid votes ($> 2/3$), nodes commit B_k to disk, update the local inventory state, and send a confirmation receipt back to the client.

Error Remediation Protocol

When an operational mistake occurs (e.g., logging 100 items inbound instead of 10), the operator initiates a compensating flow: 1. Operator creates a remediation transaction T_{remedy} targeting original transaction T_{err}. 2. T_{remedy} contains the differential delta ($\Delta\text{Qty} = -90$) and explicit references RefID = Hash(T_{err}). 3. Supervisors sign T_{remedy}. Once committed via consensus, state updates correctly without erasing T_{err}, maintaining historical auditability.

3.4 Physical Architecture and Deployment

The physical deployment model relies on containerized microservices (Docker/Kubernetes) across distributed sites:

Each regional warehouse hosts a local validator container connected to regional hardware security modules (HSM) for key management.

Client devices connect via secure REST/gRPC endpoints exposed by local API gateways.

Peer-to-peer (P2P) communication among validator containers uses encrypted TCP channels over enterprise VPN tunnels.

3.5 Development Plan

To manage implementation risk, development is divided into three incremental tiers:

Tier 1: Minimal Viable Product (MVP)
- In-memory append-only blockchain storage module.
- Single-node API server with basic CLI client for inbound/outbound logging.
- Cryptographic signature generation and SHA-256 block linking.

Tier 2: Distributed Consensus & Persistence
- Multi-node network execution with PoA consensus implementation.
- Disk persistence engine (LevelDB/RocksDB) for state recovery.
- Compensating transaction protocol for error remediation.

Tier 3: Advanced Features & Fault Injection
- Web-based dashboard interface for warehouse operators.
- Role-Based Access Control (RBAC) certificate verification module.
- Simulated Byzantine fault injection framework for robustness testing under network partition and node crash conditions.

Chapter 4

Implementation

[Implementation details regarding platform selection, specific data structures, middleware bindings, and codebase organization will be detailed here in future revisions.]

Chapter 5

Validation

[Performance benchmarks, TPS throughput graphs, fault tolerance simulation results, and UI screenshots validating the requirements of Chapter **??** *will be presented here.]*

Chapter 6

Conclusions

[Summary of achieved milestones, comparison against original objectives, and proposed future work.]

Appendix A

Source Code and Setup Instructions

[Build instructions, environment configuration scripts, and key code snippets.]